

#MEDICAL IMAGE QUESTION ANSWERING CLASSIFICATION USING
PYTORCH, TORCH VISION, TRANSFORMERS (BERT AND ViT), openCV,
MED-VQA_MODEL IMPLEMENTS A DUAL-ENCODER TRANSFORMER
ARTITECHTURE AND FUSE THEM TOGETHER.

SOURCE CODE:

```
```python
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms
from transformers import BertTokenizer, BertModel, ViTModel
from PIL import Image
import pandas as pd
import numpy as np
import cv2
import matplotlib.pyplot as plt
import os
```

```
=====
```

## # 1. CONFIGURATION

# =====

class Config:

DEVICE = "cuda" if torch.cuda.is\_available() else "cpu"

IMG\_SIZE = 224

BATCH\_SIZE = 8

EPOCHS = 3 # Increase to 20 for real training

LR = 2e-5

MAX\_TEXT\_LEN = 32

HIDDEN\_DIM = 768

NUM\_CLASSES = 5 # Small number for mock demo; use 500+ for real data

# Models

BERT\_NAME = 'bert-base-uncased'

VIT\_NAME = 'google/vit-base-patch16-224'

print(f"✅ Running on device: {Config.DEVICE}")

# =====

## # 2. MOCK DATA GENERATOR (For Testing)

# =====

def generate\_mock\_data(num\_samples=20):

"""Generates dummy images and questions so code runs immediately."""

os.makedirs("mock\_data", exist\_ok=True)

data = []

vocab = {'yes': 0, 'no': 1, 'left': 2, 'right': 3, 'normal': 4}

```

print(f"🌀 Generating {num_samples} mock samples...")
for i in range(num_samples):
 img_path = f"mock_data/img_{i}.jpg"
 # Create random noise image
 random_color = np.random.randint(0, 255, (224, 224, 3),
dtype=np.uint8)
 Image.fromarray(random_color).save(img_path)

 data.append({
 'image_path': img_path,
 'question': "Is there a finding?" if i % 2 == 0 else "Where is the
finding?",
 'answer': np.random.choice(list(vocab.keys()))
 })

return pd.DataFrame(data), vocab
=====

3. DATASET CLASS
=====

class MedVQADataset(Dataset):
 def __init__(self, df, tokenizer, transform, answer_map):
 self.df = df
 self.tokenizer = tokenizer
 self.transform = transform
 self.answer_map = answer_map

 def __len__(self):

```

```

 return len(self.df)

def __getitem__(self, idx):
 row = self.df.iloc[idx]

 # Load Image
 image = Image.open(row['image_path']).convert('RGB')
 image = self.transform(image)

 # Process Question
 encoding = self.tokenizer(
 str(row['question']),
 max_length=Config.MAX_TEXT_LEN,
 padding='max_length',
 truncation=True,
 return_tensors='pt'
)

 # Process Answer
 label = self.answer_map.get(str(row['answer']).lower(), 0)

 return {
 'image': image,
 'input_ids': encoding['input_ids'].squeeze(0),
 'attention_mask': encoding['attention_mask'].squeeze(0),
 'label': torch.tensor(label, dtype=torch.long)
 }

```

```

=====
4. MODEL ARCHITECTURE
=====

class MedVQA_Model(nn.Module):
 def __init__(self, num_classes):
 super(MedVQA_Model, self).__init__()

 # Vision Encoder (ViT)
 self.vit = ViTModel.from_pretrained(Config.VIT_NAME)

 # Text Encoder (BERT)
 self.bert = BertModel.from_pretrained(Config.BERT_NAME)

 # Cross-Attention Fusion
 self.cross_attn =
nn.MultiheadAttention(embed_dim=Config.HIDDEN_DIM,
num_heads=8, batch_first=True)

 self.layer_norm = nn.LayerNorm(Config.HIDDEN_DIM)

 # Classifier
 self.classifier = nn.Sequential(
 nn.Linear(Config.HIDDEN_DIM, Config.HIDDEN_DIM // 2),
 nn.ReLU(),
 nn.Dropout(0.1),
 nn.Linear(Config.HIDDEN_DIM // 2, num_classes)
)

 def forward(self, images, input_ids, attention_mask):

```

```

1. Vision Features [Batch, 197, 768]
vit_out = self.vit(pixel_values=images)
img_feats = vit_out.last_hidden_state

2. Text Features [Batch, Seq_Len, 768]
bert_out = self.bert(input_ids=input_ids,
attention_mask=attention_mask)
text_feats = bert_out.last_hidden_state

3. Cross-Attention Fusion (Text attends to Image)
attn_out, attn_weights = self.cross_attn(
 query=text_feats,
 key=img_feats,
 value=img_feats
)

Residual + Norm + Pooling (CLS token)
fused_feats = self.layer_norm(text_feats + attn_out)
cls_token_out = fused_feats[:, 0, :]

4. Classification
logits = self.classifier(cls_token_out)

return logits, attn_weights

=====

5. TRAINING FUNCTION

=====

def train_engine():

```

```

Setup Data
df, answer_map = generate_mock_data()
inv_vocab = {v: k for k, v in answer_map.items()}

transform = transforms.Compose([
 transforms.Resize((Config.IMG_SIZE, Config.IMG_SIZE)),
 transforms.ToTensor(),
 transforms.Normalize(mean=[0.5]*3, std=[0.5]*3)
])

tokenizer = BertTokenizer.from_pretrained(Config.BERT_NAME)
dataset = MedVQADataset(df, tokenizer, transform, answer_map)
dataloader = DataLoader(dataset, batch_size=Config.BATCH_SIZE,
shuffle=True)

Setup Model
model =
MedVQA_Model(num_classes=len(answer_map)).to(Config.DEVICE)
optimizer = optim.AdamW(model.parameters(), lr=Config.LR)
criterion = nn.CrossEntropyLoss()

print("\n🚀 Starting Training...")
model.train()

for epoch in range(Config.EPOCHS):
 total_loss = 0
 for batch in dataloader:
 imgs = batch['image'].to(Config.DEVICE)

```

```

 ids = batch['input_ids'].to(Config.DEVICE)
 mask = batch['attention_mask'].to(Config.DEVICE)
 labels = batch['label'].to(Config.DEVICE)

 optimizer.zero_grad()
 logits, _ = model(imgs, ids, mask)
 loss = criterion(logits, labels)
 loss.backward()
 optimizer.step()
 total_loss += loss.item()

 print(f" Epoch {epoch+1}/{Config.EPOCHS} | Loss:
 {total_loss/len(dataloader):.4f}")

 return model, tokenizer, inv_vocab, transform
=====

6. INFERENCE & VISUALIZATION
=====

def predict(model, img_path, question, tokenizer, vocab, transform):
 model.eval()

 # Prepare Inputs
 raw_img = Image.open(img_path).convert('RGB')
 img_tensor = transform(raw_img).unsqueeze(0).to(Config.DEVICE)

 inputs = tokenizer(question, return_tensors="pt",
padding='max_length', max_length=Config.MAX_TEXT_LEN)
 ids = inputs['input_ids'].to(Config.DEVICE)

```



```

mask = inputs['attention_mask'].to(Config.DEVICE)

Forward Pass
with torch.no_grad():
 logits, attn_weights = model(img_tensor, ids, mask)
 pred_idx = torch.argmax(logits, dim=1).item()

Process Heatmap
Average attention across heads and tokens
avg_attn =
attn_weights.mean(dim=1).mean(dim=1).squeeze().cpu().numpy()
patch_attn = avg_attn[1:] # Drop CLS token
heatmap = patch_attn.reshape(14, 14) # Reshape to grid (14x14
patches = 196)

Resize to image size & colorize
heatmap_resized = cv2.resize(heatmap, (raw_img.width,
raw_img.height))
heatmap_norm = (heatmap_resized - heatmap_resized.min()) /
(heatmap_resized.max() - heatmap_resized.min())
heatmap_img = np.uint8(255 * heatmap_norm)
heatmap_color = cv2.applyColorMap(heatmap_img,
cv2.COLORMAP_JET)

Overlay
open_cv_img = cv2.cvtColor(np.array(raw_img),
cv2.COLOR_RGB2BGR)
overlay = cv2.addWeighted(open_cv_img, 0.6, heatmap_color, 0.4, 0)

```

```

Display
print(f"\n🔍 Question: {question}")
print(f"🔍 Predicted Answer: {vocab.get(pred_idx, 'Unknown')}")

plt.figure(figsize=(10,4))
plt.subplot(1,2,1); plt.imshow(raw_img); plt.title("Original X-ray")
plt.subplot(1,2,2); plt.imshow(cv2.cvtColor(overlay,
cv2.COLOR_BGR2RGB)); plt.title("AI Attention Map")
plt.show()

=====

7. MAIN EXECUTION

=====

if __name__ == "__main__":
 # 1. Train
 model, tokenizer, vocab, transform = train_engine()

 # 2. Test
 test_img = "mock_data/img_0.jpg"
 test_q = "Is there a finding?"
 predict(model, test_img, test_q, tokenizer, vocab, transform)

```